

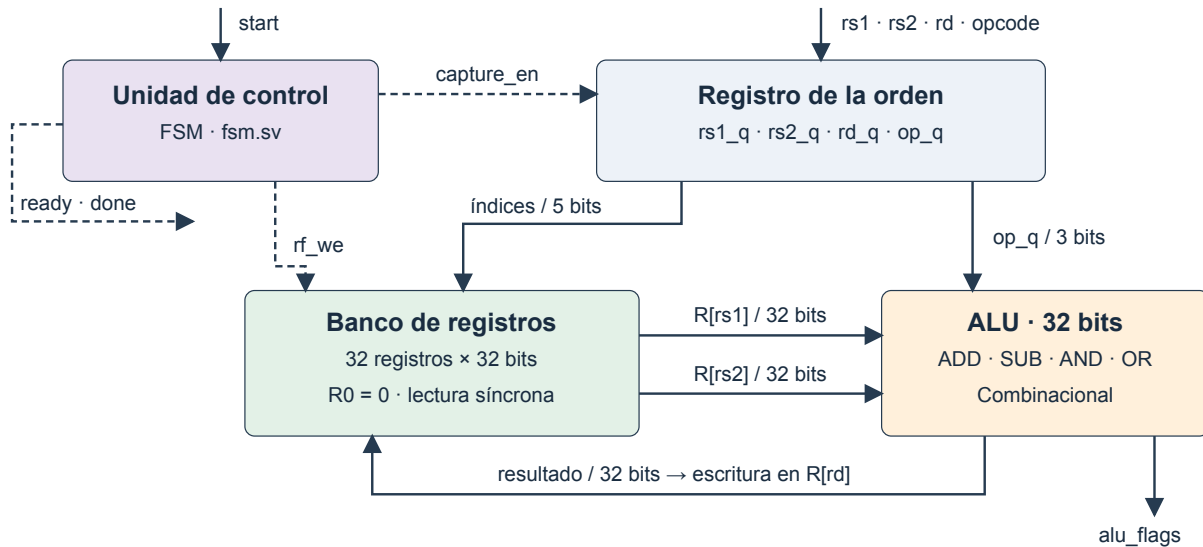
# Trabajo Práctico 1

ALU, banco de registros y unidad de control

## Descripción del problema

Implementar una microarquitectura que comunique tres módulos: **ALU**, **banco de registros** y **unidad de control**. Un registro auxiliar conserva la orden aceptada. La operación es  $R[rd] \leftarrow R[rs1] \text{ op } R[rs2]$ :  $rd$  identifica el registro destino;  $rs1$  y  $rs2$ , los registros fuente;  $op$  puede ser ADD, SUB, AND u OR.

La ALU opera con **datos de 32 bits**. El banco contiene **32 registros de 32 bits**: R1 a R31 son de propósito general; R0 se lee siempre como cero y descarta escrituras. Los índices  $rs1$ ,  $rs2$  y  $rd$  tienen **5 bits**; no son los operandos.



clk y rst: control, registro de orden y banco. Líneas discontinuas: señales de control.

Figura 1: Esquema funcional de la unidad de ejecución. El mux del índice A se detalla en el ejercicio 3.

La **unidad completa** recibe  $clk$ ,  $rst$ ,  $start$ ,  $rs1$ ,  $rs2$ ,  $rd$  y  $opcode$ ; produce  $ready$ ,  $done$  y  $alu\_flags$ . La ALU es combinacional: recibe dos operandos y un opcode; entrega resultado y flags. El resultado se escribe en el banco y no es una salida externa de la unidad.

$rst$  es asíncrono, activo en alto: reinicia el control, la orden capturada y las salidas de lectura del banco; no borra su contenido. Se asume que no se solicitan operaciones durante reset.

Una orden se acepta en un flanco ascendente con  $ready=1$  y  $start=1$ . Los cuatro campos deben estar estables en ese flanco; luego pueden cambiar. Mientras está ocupada, la unidad ignora nuevas órdenes.  $done=1$  identifica el ciclo WRITEBACK: resultado y flags son válidos durante ese ciclo; la escritura ocurre en el flanco que lo finaliza. Después vuelve a IDLE, con  $ready=1$  y  $done=0$ . Las flags no se conservan necesariamente después.

## Archivos provistos y requisitos

tp1\_pkg.sv contiene las definiciones compartidas. import tp1\_pkg::\*; permite usar sus nombres:

| Definición      | Significado                                          |
|-----------------|------------------------------------------------------|
| DATA_WIDTH = 32 | Ancho de los datos y del resultado                   |
| ADDR_WIDTH = 5  | Ancho de los índices: 32 registros                   |
| alu_op_e        | Tipo de opcode: OP_ADD, OP_SUB, OP_AND, OP_OR        |
| alu_flags_t     | Estructura con campos z, n, c, v, de un bit cada uno |

alu\_if.sv agrupa las señales de la ALU: operand\_a, operand\_b, opcode, result y flags. Por ejemplo, alu\_io.flags.z accede a Z. bloques\_alu.sv contiene los módulos aritméticos y detectores del ejercicio 1.

Completar los módulos indicados, respetando sus puertos, tipos y comportamiento. La organización interna puede variar; se deben reutilizar los bloques provistos. No modificar esos bloques ni los testbenches. En cada carpeta:

```
make sim      # actualiza dependencias, compila y ejecuta las pruebas
make wave     # vuelve a simular y abre obj_dir/sim.vcd
```

Editar cada módulo en su ejercicio de origen: las copias se sobrescriben. **No se requiere agregar archivos ni editar los Makefiles.** El paquete incluye las fuentes y dependencias necesarias. La simulación no requiere Python. Las pruebas de cada ejercicio deben finalizar con PASS.

## Ejercicio 1 – ALU y flags

Completar ejercicios/ej1/alu.sv mediante instancias de los módulos provistos en bloques\_alu.sv. No reimplementar la suma, la resta ni los detectores de Z/N. Los cuatro módulos están agrupados en un archivo, parametrizados por DATA\_WIDTH e instanciados con ancho 32. No requieren sumadores de un bit.

- sumador\_flags: entradas a, b; salidas sum, carry y overflow.
- restador\_flags: entradas a, b; salidas resta, carry y overflow.
- comparador: entradas a, b de 32 bits; salida iguales, que vale 1 si a=b.
- negativo: entrada dato; salida negativo, igual al bit más significativo (bit 31).

Conectar los operandos al sumador y al restador. Seleccionar el resultado y C/V según el opcode; para AND/OR usar &/| y C=V=0. Conectar el resultado seleccionado a una entrada del comparador y cero a la otra para obtener Z. Conectar ese mismo resultado al detector negativo para obtener N. El comparador y el detector de signo se comparten entre las cuatro operaciones.

La ALU recibe operandos de 32 bits y opcode de 3 bits mediante alu\_io; entrega alu\_io.result y alu\_io.flags. No se requieren otros componentes para las operaciones lógicas. Respetar la excepción de flags del opcode inválido.

| Opcode  | Operación | Resultado                      |
|---------|-----------|--------------------------------|
| 000     | ADD       | A + B                          |
| 001     | SUB       | A – B                          |
| 010     | AND       | AND bit a bit                  |
| 011     | OR        | OR bit a bit                   |
| 100–111 | Inválido  | Cero; flags Z=0, N=1, C=0, V=0 |

Para operaciones válidas, Z indica resultado cero y N es el bit 31 del resultado. En ADD, C indica acarreo fuera de los 32 bits. En SUB, C indica préstamo unsigned ( $A < B$ ). V indica que el resultado exacto en complemento a dos está fuera de  $[-2^{31}, 2^{31} - 1]$ . Para opcode inválido se conserva la convención de la tabla, aun cuando no coincida con el significado habitual de Z/N.

Calcular resultado de 32 bits y Z/N/C/V para 10+20, 10–20, 10–10, 32'hFFFF\_FFFF+1, 32'h7FFF\_FFFF+1 y 32'h8000\_0000–1. Interpretar los operandos en complemento a dos para evaluar V. Contrastar con la simulación.

## Ejercicio 2 — Banco de registros

El módulo `ejercicios/ej2/reg_file.sv` está provisto. Cada puerto A/B tiene un índice `regX_idx`, un dato de entrada `regX_din`, uno de salida `regX_dout` y una habilitación de escritura `regX_we`.

En cada flanco ascendente, la salida de cada puerto toma el contenido de su índice. Entre flancos conserva ese valor. Si `we=1`, se escribe `din` en ese mismo índice. La lectura es **read-first**: si se lee y escribe una posición en el mismo flanco, `dout` recibe el contenido anterior. No se utilizan escrituras simultáneas de ambos puertos a una misma posición distinta de R0.

Completar la tabla antes de simular. Se parte de `R1=10`, `R2=20` y de una lectura ya realizada con `A=1` y `B=2`. Salvo indicación, ambas escrituras están deshabilitadas.

| Instante / acción                          | dout A | dout B | R2 |
|--------------------------------------------|--------|--------|----|
| Situación inicial                          |        |        |    |
| A cambia a índice 2, antes del flanco      |        |        |    |
| Después del siguiente flanco               |        |        |    |
| Después de escribir 99 en R2 por A         |        |        |    |
| Después del siguiente flanco sin escritura |        |        |    |

Ejecutar `make sim` y contrastar con `make wave`. Explicar qué ocurre al intentar escribir R0. No modificar código en este ejercicio.

## Ejercicio 3 — Camino de datos

Completar `ejercicios/ej3/datapath.sv`, que instancia la ALU del ejercicio 1 y el banco del ejercicio 2. Los datos y resultados tienen 32 bits; los índices, 5. El testbench mantiene los índices y el opcode estables y controla la señal `rf_we`; todavía no se utiliza una FSM.

Conectar:

- `alu_io.operand_a` y `alu_io.operand_b` a `rf_rs1_data` y `rf_rs2_data`.
- `result` y `flags` a las salidas de la ALU.
- `regA_idx` a `rs1` cuando `rf_we=0`, y a `rd` cuando `rf_we=1`.

La conexión del opcode y las instancias están provistas. El puerto B lee `rs2` y no escribe. El puerto A recibe `result` como dato de escritura.

Verificar  $R5 \leftarrow R1 + R2$  con `R1=10` y `R2=20`, y luego  $R6 \leftarrow R5 - R1$ . El resultado debe quedar escrito en el destino sin alterar los orígenes, salvo que el destino coincida con uno de ellos.

En la traza, identificar cuándo se actualizan los operandos y cuándo se escribe el destino. Explicar por qué seleccionar `rd` no cambia inmediatamente el operando A de la ALU.

## Ejercicio 4 — Registro de la orden

Completar `ejercicios/ej4/registro_orden.sv`. En cada flanco ascendente con `capture_en=1`, guardar `rs1`, `rs2`, `rd` y opcode en `rs1_q`, `rs2_q`, `rd_q` y `op_q`. Con `capture_en=0`, conservarlos. El reset está provisto.

La prueba debe confirmar que los campos guardados no cambian al modificar las entradas con la captura deshabilitada, y que una nueva captura los reemplaza.

## Ejercicio 5 — Unidad de control

Completar próximo estado y salidas en ejercicios/ej5/fsm.sv según el diagrama. El registro de estado está provisto. El estado inicial es IDLE; con start=0 permanece allí. Las transiciones sin etiqueta son incondicionales y se realizan en el siguiente flanco ascendente.

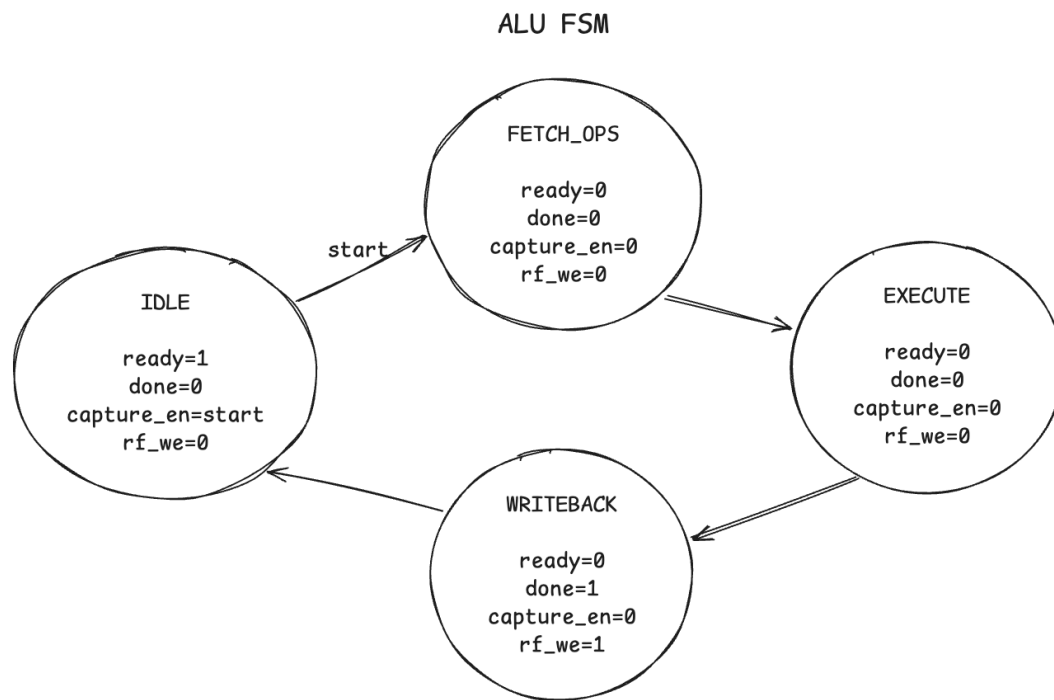


Figura 2: Estados y salidas de la unidad de control.

capture\_en se activa combinatorialmente en IDLE cuando start=1. rf\_we habilita la escritura durante WRITEBACK. El banco escribe en el flanco que termina ese estado, al mismo tiempo que la FSM vuelve a IDLE. Un start que dure dos ciclos se acepta una sola vez si comienza en IDLE; si permanece activo hasta el siguiente IDLE, puede iniciar otra operación.

Verificar espera en IDLE, secuencia completa, captura, escritura y reset. No debe aceptarse ni quedar pendiente una orden recibida mientras está ocupado.

## Ejercicio 6 — Unidad de ejecución completa

Completar las conexiones en ejercicios/ej6/top\_module.sv:

- La FSM recibe start y produce ready, done, capture\_en y rf\_we.
- El registro de orden recibe las entradas externas y capture\_en.
- El datapath recibe los campos guardados y rf\_we; entrega alu\_flags.
- La FSM, el registro de orden y el banco comparten clk y rst; la ALU no los recibe.

El testbench precarga el banco. Deben pasar las pruebas de operaciones, flags, escritura efectiva, dependencia entre operaciones, destino igual a origen, R0 y cambios de las entradas externas mientras la unidad está ocupada.

Para  $R5 \leftarrow R1 + R2$  con  $R1=10$  y  $R2=20$ , entregar una captura de la traza con clk, start, ready, done, estado, capture\_en, rf\_we, regA\_idx, rf\_rs1\_data y result. Señalar los flancos de captura, lectura y escritura. Indicar los valores de regA\_idx y result durante FETCH\_OPS, EXECUTE y WRITEBACK, y explicar por qué el resultado se obtiene antes de escribirse.

## Entrega

Ejecutar `make sim` para verificar el trabajo y luego `make entrega` desde `tp/`, `ejercicios/` o cualquier carpeta `ejN/`. Se genera `tp/entrega/` con los cinco módulos propios, conservando su carpeta de ejercicio: `ej1/alu.sv`, `ej3/datapath.sv`, `ej4/registro_orden.sv`, `ej5/fsm.sv` y `ej6/top_module.sv`. No se incluyen testbenches, componentes provistos, Makefiles ni compilados.

Preparar **un único PDF**, `tp/respuestas/respuestas.pdf`, con:

- Nombres e identificación de los integrantes.
- Ejercicio 1: cuentas de resultado y flags.
- Ejercicio 2: tabla y explicación sobre R0.
- Ejercicio 3: explicación sobre el índice A y la lectura síncrona.
- Ejercicio 6: captura de la traza con los flancos señalados y respuestas solicitadas.

Incluir las capturas dentro del PDF y ejecutar nuevamente `make entrega` para copiarlo. Entregar la carpeta **entrega/ completa**, incluyendo `respuestas/`. El comando avisa si falta el PDF, pero no verifica su contenido ni ejecuta tests. Cada integrante debe poder explicar el diseño en el coloquio.